

PWM ile Akım Kontrolü

Hedef artık RPM değil, amper.

Bu rehberde yazılımın görevi, örneğin 2 A istenen akıma ulaşmak ve bu akımı korumak. Kontrolcü ölçülen akıma bakacak; PWM'in açık kalma oranını artırıp azaltacak. Önce devreyi Python ile taklit edecek, ardından davranışı grafiklerde inceleyeceğiz.

İlk çalışan örnek

12 V besleme, 2 Ω direnç ve 2 mH endüktans. Her 100 μ s'de bir kontrol hesabı. Hedef 2 A. Başlangıç ayarları: $K_p = 0.10$, $K_i = 100$, $K_d = 0$.

$K_d = 0$ olduğunda PID kodu PI kontrolcüsü olarak çalışır. Bu basit akım devresinde önce PI'yi anlayıp kullanacağız.

Okuma sırası

01-07 Devre, PWM, formüller ve sayısal hesap.

12-16 Beş deney ve grafiklerin yorumu.

Önceki hız örneğinden ne değişti?

	HIZ KONTROLÜ	BU REHBER
Hedef	İstenen RPM	İstenen akım, A
Geri besleme	Ölçülen RPM	Ölçülen akım, A
Hata	RPM farkı	Amper farkı
Kontrol çıkışı	PWM duty	PWM duty
Taklit edilen sistem	Mekanik hız cevabı	Elektriksel RL cevabı

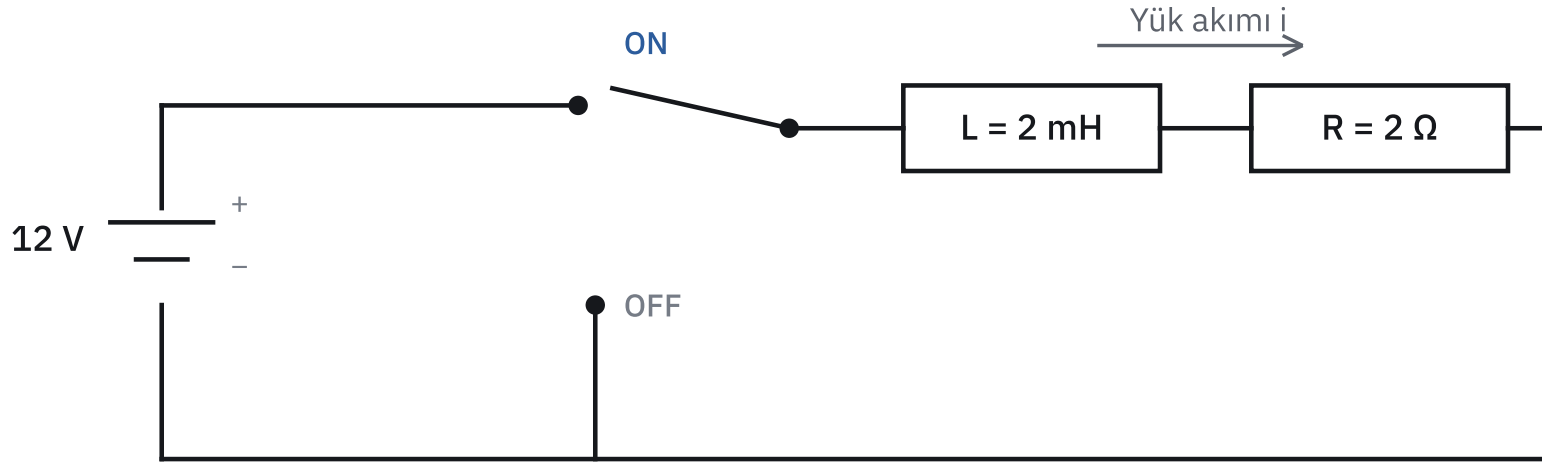
08-11 Python kodunu kurma, anlama ve çalıştırma.

17-19 Donanıma geçiş, alıştırmalar ve kaynaklar.

Örnek değerler eğitim için seçilmiştir. Çıktılar Python simülasyonundan üretilmiştir; fiziksel devre ölçümü değildir. Paket, PDF'nin düzenlenebilir LaTeX kaynağını da içerir.

Kontrolcü hangi devreyi kontrol ediyor?

Seri RL yükü kullanıyoruz: R akımın karşılaştığı direnci, L ise akımın ani değişimine karşı koyan endüktansı temsil eder. Besleme sabit 12 V'tur.



Diyagramı oku: Anahtar yükün sol ucunu ya 12 V'a ya da dönüş hattına bağlar. OFF durumunda akımın dolaşabileceği yol vardır. Bu, ideal serbest dolaşım davranışını temsil eden bir işlev şemasıdır; MOSFET sürücü bağlantı şeması değildir.

İdeal anahtar seçimi

- **ON:** yük 12 V'a bağlı
- **OFF:** akım kapalı yolda dolaşır



← Akım ölçümü Python: model

Hedefi değiştiren sensör değildir

Sen `target_current = 2.0` yazarsın. Kontrolcü, ölçümle hedefi karşılaştırıp duty hesaplar. Devre akımı bu duty'ye fizik kurallarıyla tepki verir.

Duty akımı nasıl değiştirir?

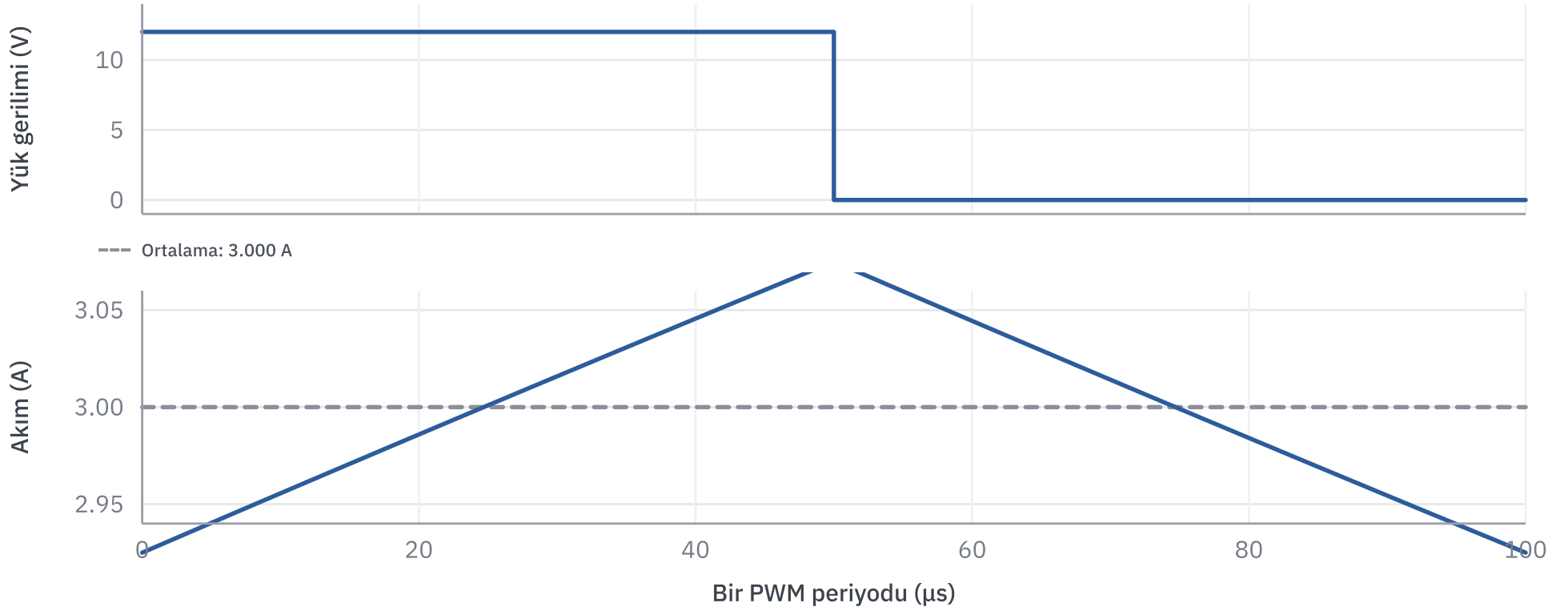
Duty d , bir periyodun ne kadarında ON durumunda kaldığımızdır:

$$d = \frac{T_{on}}{T_{PWM}}, \quad f_{PWM} = 10\,000 \text{ Hz}, \quad T_{PWM} = 100 \mu\text{s}.$$

DUTY D	ON SÜRESİ	ORTALAMA GERİLİM	DENGE AKIMI
0.25 (%25)	25 μs	3 V	1.5 A
0.50 (%50)	50 μs	6 V	3 A
0.75 (%75)	75 μs	9 V	4.5 A

Tablo bu ideal devrenin $R = 2 \Omega$ durumuna aittir. Kullanılan ilişkiler:

$$v = d V_{bus}, \quad i_{denge} = \frac{v}{R} = \frac{12d}{2} = 6d.$$



Grafiği oku: İlk 50 μs boyunca gerilim 12 V ve akım yükseliyor. Sonraki 50 μs 'de yük gerilimi 0 V ve akım azalıyor. Akım, anahtar OFF olduğunda aniden sıfıra düşmüyor. Endüktansta depolanan enerji dolaşım yolundaki akımı sürdürüyor.

Ortalama 3 A, anlık akım tam sabit değil

Yerleşmiş durumda yaklaşık 2.925–3.075 A arasında dalgalanma var. Bu örnekte tepeden tepeye dalgalanma yaklaşık 0.150 A. Kontrol hedefimiz ortalama akım davranışıdır; PWM dalgalanmasını ayrıca inceleyeceğiz.

Eski “0.20” katsayısı yerine fizik

Bir seri RL devresinde uygulanan gerilim, direnç ve endüktans üzerindeki gerilimlerin toplamıdır:

$$v = Ri + L \frac{di}{dt} \implies \frac{di}{dt} = \frac{v - Ri}{L}.$$

Ri mevcut akımın dirençte oluşturduğu gerilim düşümüdür. $v > Ri$ ise akım artar; $v < Ri$ ise azalır. $v = Ri$ olduğunda değişim durur.

Python’da bir sonraki akımı nasıl buluyoruz?

Bir adım boyunca v , R ve L sabitse tam analitik çözüm:

$$i_{k+1} = i_k + \alpha (i_{denge} - i_k), \quad i_{denge} = \frac{d_k V_{bus}}{R}, \quad \alpha = 1 - e^{-R\Delta t/L}, \quad \tau = \frac{L}{R}.$$

Bizde $\tau = 0.002/2 = 0.001$ s ve $\Delta t = 0.0001$ s:

$$\alpha = 1 - e^{-0.1} \approx 0.095163.$$

Yani bu adımda denge akımına kalan farkın yaklaşık %9.516’sı kapanır. Katsayı rastgele seçilmedi; devre değerleri ve zaman aralığından hesaplandı.

Somut hesap: duty %50, mevcut akım 1 A

$$i_{denge} = \frac{0.5 \times 12}{2} = 3 \text{ A}, \quad i_{k+1} = 1 + 0.095163 \times (3 - 1) \approx 1.190325 \text{ A}.$$

i_{k+1} yeni akım 1 eski akım 0.095163 tepki oranı (3 - 1) kalan fark

Mevcut akım 4 A olsaydı aynı duty için sonuç $4 + 0.095163(3 - 4) = 3.904837$ A olurdu: akım azalırdı.

İki farklı “istenen” kavramını ayır

i_{ref} senin hedefindir. i_{denge} ise o anki duty sabit tutulursa devrenin yaklaşacağı akımdır. Kontrolcü duty’yi değiştirdikçe i_{denge} de değişir. Bu devre denklemi PID hesabının kendisi değildir.

Formüldeki her sembolün kod karşılığı

Kontrol adımını k olsun. Ölçülen akımı i_k , hedefi r_k ile gösterelim. İntegral katkısını akımla karıştırmamak için J_k yazacağız.

$e_k = r_k - i_k$	hata, A
$P_k = K_p e_k$	anlık hata katkısı
$J_k^* = J_{k-1} + K_i e_k \Delta t$	aday integral katkısı
$q_k = \frac{i_k - i_{k-1}}{\Delta t}$	ölçülen değişim, A/s
$\beta = \frac{\Delta t}{\tau_D + \Delta t}, \quad q_{f,k} = q_{f,k-1} + \beta(q_k - q_{f,k-1})$	türev filtresi
$D_k = -K_d q_{f,k}$	filtreli, ölçüm üzerinden türev
$d_k = \text{clip}(P_k + J_k + D_k, 0, 1)$	PWM duty

J_k , anti-windup kontrolünden geçen integral katkısıdır; doyum yoksa $J_k = J_k^*$. İlk ölçümde önceki akım mevcut ölçüme eşitlenir, böylece yapay ilk türev sıçraması oluşmaz.

KOD DEĞİŞKENİ	ANLAMI	BİRİM
target , measured	Hedef ve ölçülen akım	A
error	Hedef eksi ölçüm	A
p_term , i_term , d_term	Çıkışa yapılan katkılar	Duty
kp	P katsayısı	A^{-1}
ki	I katsayısı	$(A\ s)^{-1}$
kd	D katsayısı	s/A
dt , tau_d	Zaman aralığı, filtre süresi	s

Bu rehberde duty 0–1 aralığında

duty = 0.22 , %22 PWM demektir. p_term = 0.20 , 20 yüzde puanlık katkıdır. Eski RPM örneğindeki 0–100 ölçeğini veya kazançları buraya doğrudan taşıma.

```
duty = p_term + i_term + d_term
# duty += ... yazmıyoruz: cikis her adimda yeniden hesaplanir.
```

2 A hedefine ilk iki adım

$K_p = 0.10$, $K_i = 100$, $K_d = 0$, $\Delta t = 0.0001$ s. Başlangıçta akım ve integral katkısı sıfır. Bu hesapta çıkış sınıra ulaşmadığı için anti-windup devreye girmiyor.

1. adım: önce kontrolcü

$$e = 2 - 0 = 2 \text{ A}$$

$$P = 0.10 \times 2 = 0.20$$

$$J = 0 + 100 \times 2 \times 0.0001 = 0.02$$

$$d = P + J = 0.22 \implies 22\%$$

Sonra devre bu PWM'e tepki verir

$$v = 0.22 \times 12 = 2.64 \text{ V}$$

$$i_{denge} = 2.64/2 = 1.32 \text{ A}$$

$$i_{yeni} = 0 + 0.095163(1.32 - 0) = 0.125615 \text{ A}$$

2. adım: önceki integral unutulmaz

$$e = 2 - 0.125615 = 1.874385$$

$$P = 0.187439$$

$$J = 0.02 + 100 \times 1.874385 \times 0.0001 = 0.038744$$

$$d = 0.226182$$

$$i_{denge} = 6d = 1.357094$$

$$i_{yeni} \approx 0.242805 \text{ A}$$

J'deki 0.02 önceki adımın katkısıdır.

Neden bir anda 2 A olmadı? Kontrolcünün ilk duty'si %22; devre de endüktansı nedeniyle kademeli cevap veriyor. Sonraki adımda yeniden ölçüp hesaplayacağız.

ADIM	ÖNCEKİ A	P	J	DUTY	YENİ A
1	0.000000	0.200000	0.020000	0.220000	0.125615
2	0.125615	0.187439	0.038744	0.226182	0.242805
3	0.242805	0.175719	0.056316	0.232035	0.352186
4	0.352186	0.164781	0.072794	0.237575	0.454321
5	0.454321	0.154568	0.088251	0.242819	0.549730

Değerler Python çıktısından alınmış ve altı basamağa yuvarlanmıştır. Tam hassasiyetli sonuçlar **analiz.json** içindedir.

Hata sıfırken akımı kim sürdürüyor?

2 A akımı 2 Ω üzerinden sürdürmek için ortalamada 4 V gerekir:

$$v = Ri = 2 \times 2 = 4 \text{ V}, \quad d = 4/12 = 0.333333.$$

Dengede duty yaklaşık %33.33 olmalı. Hata sıfıra geldiğinde P katkısı sıfırdır. Akım değişmiyorsa D de sıfırdır. Bu gerekli duty'yi integral katkısı taşır:

$$e = 0, \quad P = 0, \quad D = 0, \quad J \approx 0.333333.$$

Integral akımın kendisi değildir

`i_term` isminde “i” var diye amper sanma. Bu değişken integral teriminin duty katkısını saklar. Hata sıfırsa katkı sıfırlanmaz; bir sonraki artış sıfır olur.

Yalnız P olsaydı?

$d = 0.1(2 - i)$ ve devre dengesi $i = 6d$:

$$i = 6 \times 0.1(2 - i) = 1.2 - 0.6i$$

$$1.6i = 1.2 \implies i = 0.75 \text{ A}$$

P, akımı sürdürecektir bir çıkış üretmek için hataya ihtiyaç duyar. Bu ayarla 2 A hedefinden 1.25 A aşağıda kalır.

Duty'yi baştan %33.33 yapsak olmaz mı?

Bu devrenin tam 12 V ve 2 Ω olduğunu biliyorsan, sabit duty de 2 A'ya götürür. Fakat direnç 3 Ω olursa:

$$i = \frac{(1/3) 12}{3} = 1.333333 \text{ A}.$$

2 A'yı korumak için yeni duty $2 \times 3/12 = 0.50$ olmalıdır. Kapalı çevrim, akımı ölçüp bu düzeltmeyi yapar.

Kontrolcü fiziksel sınırı aşamaz

12 V ve 2 Ω ile bu modelin en büyük denge akımı 6 A'dır. 8 A hedefi için $d = 8 \times 2/12 = 1.333$ gerekir. Duty 1'i aşamayacağı için hedef ulaşamaz.

0.10 ve 100 nereden geliyor?

Bu sayılar bütün akım devreleri için sabit değildir. Burada ideal RL modelinden anlaşılır bir başlangıç ayarı çıkarıyoruz. Ana deneyde $K_d = 0$.

Denklemleri takip etmek istersen

Duty'den akıma sürekli zaman modeli ve paralel PI:

$$G(s) = \frac{V_{bus}}{Ls + R}, \quad C(s) = K_p + \frac{K_i}{s}.$$

s sürekli zaman analizindeki Laplace değişkenidir; Python kodunda hesaplamana gerek yoktur. $K_i/K_p = R/L$ seçersek, ideal modelin kutbu ile PI'nin sıfırı birbirini götürür.^[1]

$$K_p = \frac{L \omega_c}{V_{bus}}, \quad K_i = \frac{R \omega_c}{V_{bus}}.$$

Örnek tasarım seçimi olarak $\omega_c = 600$ rad/s alalım:

$$K_p = \frac{0.002 \times 600}{12} = 0.10, \quad K_i = \frac{2 \times 600}{12} = 100.$$

İdeal sürekli zaman kapalı çevrim yaklaşık $\omega_c/(s + \omega_c)$ olur. Burada $f_c = \omega_c/(2\pi) \approx 95.5$ Hz'dir; bu sayı 10 kHz PWM frekansından farklıdır.

Formüller çıkış ölçeğine bağlıdır

Bizim PI çıkışımız duty olduğu için formülde V_{bus} 'a bölüyoruz. PI çıkışını volt olarak tanımlasaydık kazançların birimleri ve sayısal değerleri değişirdi. Aynı sayıları farklı ölçeğe doğrudan kopyalamak yanlış olur.

Bu ayarın sınırları

Hesap ideal, sürekli zamanlı RL yükü içindir. Python'daki örnekleme, integral güncellemesi ve PWM periyot ortalamasının gecikmesi küçük farklar oluşturur. Donanımda sensör filtresi, hesap gecikmesi ve devre parametreleri de etkiler. Bu nedenle katsayılar simülasyonda incelenen başlangıç değerleridir.

Kontrollü bir değişiklik deneyi

K_p ve K_i değerlerini birlikte yarıya indir: 0.05 ve 50. Aynı hedef için daha yavaş cevap bekle. Sonra varsayılan değerlere dön. Bir defada tek değişikliği incelemek neden-sonuç ilişkisini açık tutar.

[1] Texas Instruments, Dave Wilson, *Teaching Your PI Controller to Behave, Part II*. Kaynaktaki seri PI ile buradaki paralel PI'nin katsayı isimleri aynı değildir.

Önce bu kısa örneği çalıştır

`baslangic.py` yalnız standart Python kütüphanesini kullanır. Hedef ilk adımdan itibaren 2 A'dır. Formüllerin hesap sırasını görmek için hazırlanmıştır.

```

import math

vbus = 12.0          # volt
resistance = 2.0     # ohm
inductance = 0.002   # henry
dt = 0.0001          # saniye, 100 us
kp = 0.10            # duty/A
ki = 100.0           # duty/(A*s)
target_current = 2.0 # amper

current = 0.0        # modelin mevcut akimi
i_term = 0.0         # integralin duty katkisi

print("adim, onceki_A, hata_A, P, I, duty, sonraki_A")
for k in range(1000):      # 1000 x 100 us = 0.1 s simulasyon
    error = target_current - current
    p_term = kp * error
    i_term += ki * error * dt
    duty = min(1.0, max(0.0, p_term + i_term))
    average_voltage = duty * vbus
    equilibrium_current = average_voltage / resistance
    alpha = 1.0 - math.exp(-resistance * dt / inductance)
    next_current = current + alpha * (equilibrium_current - current)
    if k < 5:
        print(k, *(round(x, 6) for x in
                      (current, error, p_term, i_term, duty, next_current)))
    current = next_current

print(f"Son akim: {current:.6f} A, duty: %{100*duty:.3f}")

```

`python baslangic.py` komutuyla çalıştır. Son satır yaklaşık olarak:

```
Son akim: 2.000000 A, duty: %33.333
```

Kısa örnekte doyum koruması yok

Duty sınırlandırılıyor, fakat integral birikmesi engellenmiyor. Bu 2 A senaryosunu öğrenmek için yeterli; hedef ve yük deneylerinde sonraki bölümdeki tam PID sınıfını kullan.

Kalıcı durum ve PID güncellemesi

`akim_kontrol.py` dosyasındaki sınıf aşağıdadır. Bir nesne oluşturup her adımda aynı nesnenin `update()` fonksiyonunu çağırıyoruz.

```

@dataclass
class PID:
    kp: float = 0.10          # duty / A
    ki: float = 100.0         # duty / (A*s)
    kd: float = 0.0          # duty*s / A; 0 -> PI
    tau_d: float = 0.0005    # turev filtresi zaman sabiti, s
    anti_windup: bool = True
    i_term: float = 0.0       # akim degil: integralin duty katkisi
    previous_current: float | None = None
    filtered_rate: float = 0.0 # A/s

    def update(self, target, measured, dt):
        if not all(math.isfinite(x) for x in (target, measured, dt)) or dt <= 0:
            raise ValueError("Hedef/olcum sonlu, dt pozitif ve sonlu olmalı")
        error = target - measured
        p_term = self.kp * error
        if self.previous_current is None:
            self.previous_current = measured
        rate = (measured - self.previous_current) / dt
        beta = dt / (self.tau_d + dt)
        self.filtered_rate += beta * (rate - self.filtered_rate)
        d_term = -self.kd * self.filtered_rate
        candidate_i = self.i_term + self.ki * error * dt
        candidate_u = p_term + candidate_i + d_term
        pushes_high = candidate_u > 1.0 and error > 0.0
        pushes_low = candidate_u < 0.0 and error < 0.0
        blocked = self.anti_windup and (pushes_high or pushes_low)
        if not blocked:
            self.i_term = candidate_i
        duty = clamp(p_term + self.i_term + d_term)
        self.previous_current = measured
        return duty, p_term, self.i_term, d_term, error

```

Kalıcı olanlar: `i_term`, `previous_current` ve `filtered_rate`. Bunlar her adımda sıfırlanırsa I ve D doğru çalışmaz. `self`, bu PID nesnesinin kendi durumunu ifade eder.

Doyum mantığı: Aday çıkış 1'in üzerindeyken hata pozitifse, integralin daha da büyümesine izin verme. Aday çıkış 0'ın altındayken hata negatifse, daha da küçülmesine izin verme. Ters yöndeki hata integralin geri dönmesine izin verir.^[2]

Çıkış yine son satırda 0–1 aralığına kırılır. Anti-windup, ulaşılmaz bir hedefi ulaşılır yapmaz; birikmeden kaynaklanan gecikmeyi azaltır.

Ölç, hesapla, devreyi ilerlet

Ortalama modelin bir adımı, gerilimi $d V_{bus}$ olarak RL denklemindeki analitik çözümü kullanır:

```
def rl_step(current, voltage, resistance, inductance, dt):
    """Sabit voltaj altında RL devresinin tam analitik zaman adımı."""
    if resistance <= 0 or inductance <= 0 or dt < 0:
        raise ValueError("R ve L pozitif, dt negatif olmayan bir sayı olmalı")
    equilibrium_current = voltage / resistance
    alpha = -math.exp(-resistance * dt / inductance)
    return current + alpha * (equilibrium_current - current)

def averaged_step(current, duty, resistance, dt=DT):
    return rl_step(current, clamp(duty) * VBUS, resistance, L_NOM, dt)
```

`-math.exp(-x)`, $1 - e^{-x}$ ile aynıdır. Küçük x değerlerinde sayısal hassasiyeti korumak için kullanılır.

Döngünün özeti

```
pid = PID()          # dongunun DISINDA, bir kez
current = 0.0
dt = 100e-6

for k in range(1600):
    t = k * dt
    target = 2.0 if k >= 50 else 0.0
    measured = current
    duty, p, j, d, error = pid.update(target, measured, dt)
    next_current = averaged_step(current, duty, 2.0, dt)
    # Burada zaman, hedef, akim, duty ve terimler kaydedilir.
    current = next_current
```

t_k : akımı oku
 i_k



PID hesabı
 d_k



Δt süre uygula
 i_{k+1} hesapla



Yeni akımı sakla
 $k \leftarrow k + 1$

← Bir sonraki kontrol adımı aynı sırayı tekrarlar.

Zamanı karıştırma

$1600 \text{ adım} \times 100 \mu\text{s} = 0.16 \text{ s}$ fiziksel simülasyon süresidir. Bilgisayarda program daha hızlı veya daha yavaş çalışabilir. `time.sleep()` kullanmıyoruz; zaman bir sayısal değişken olarak ilerliyor.

CSV okurken: `i_start_A` adımın başı, `i_next_A` adımın sonudur. Grafikte son akımın zamanı $t_k + \Delta t$ olarak çizilir. Böylece eski akımla yeni duty aynı ölçüm anıymış gibi karıştırılmaz.

Grafikleri kendi bilgisayarında üret

Python 3.10 veya üstüyle paketi bir klasöre aç. Terminali bu klasörde başlat:

```
python -m pip install -r requirements.txt
python baslangic.py
python akim_kontrol.py
```

Bazı sistemlerde komut `python3` olabilir. Ana program grafik penceresi açmaz; çıktıları dosyaların yanındaki `sonuclar/` klasörüne kaydeder.

DOSYA	NE İÇİN KULLANACAKSIN?
<code>baslangic.py</code>	İlk hesapları elle yaptığın değerlerle karşılaştır.
<code>akim_kontrol.py</code>	PID, devre modeli ve bütün deneyler.
<code>requirements.txt</code>	Gerekli NumPy ve Matplotlib paketleri.
<code>sonuclar/*.png</code> , <code>*.pdf</code>	Altı okunaklı grafik; PDF sürümleri vektörel.
<code>sonuclar/*.csv</code>	Her deneyin sayısal zaman serisi.
<code>sonuclar/analiz.json</code>	İlk beş adım ve hesaplanan sonuçlar.
<code>*.tex</code>	Bu rehberin düzenlenebilir LaTeX kaynağı.

Başarıyı hangi sayılarla ölçüyoruz?

Kalıcı hata: Hedef eksi son akım. Biriminin amper olduğuna dikkat et.

Yerleşme süresi: Hedef değişiminden sonra akımın hedefin $\pm 2\%$ bandına girip kalan simülasyon boyunca orada kalmasına kadar geçen süre. 2 A için bant 1.96–2.04 A'dır. Bandı hiç kalıcı olarak

Aşım: Hedefin üstüne çıkan en büyük miktarın hedefe oranı.
Buradaki basamak deneyinde başlangıç hedefi sıfırdır.

yakalamazsa sonuç `null` olur.

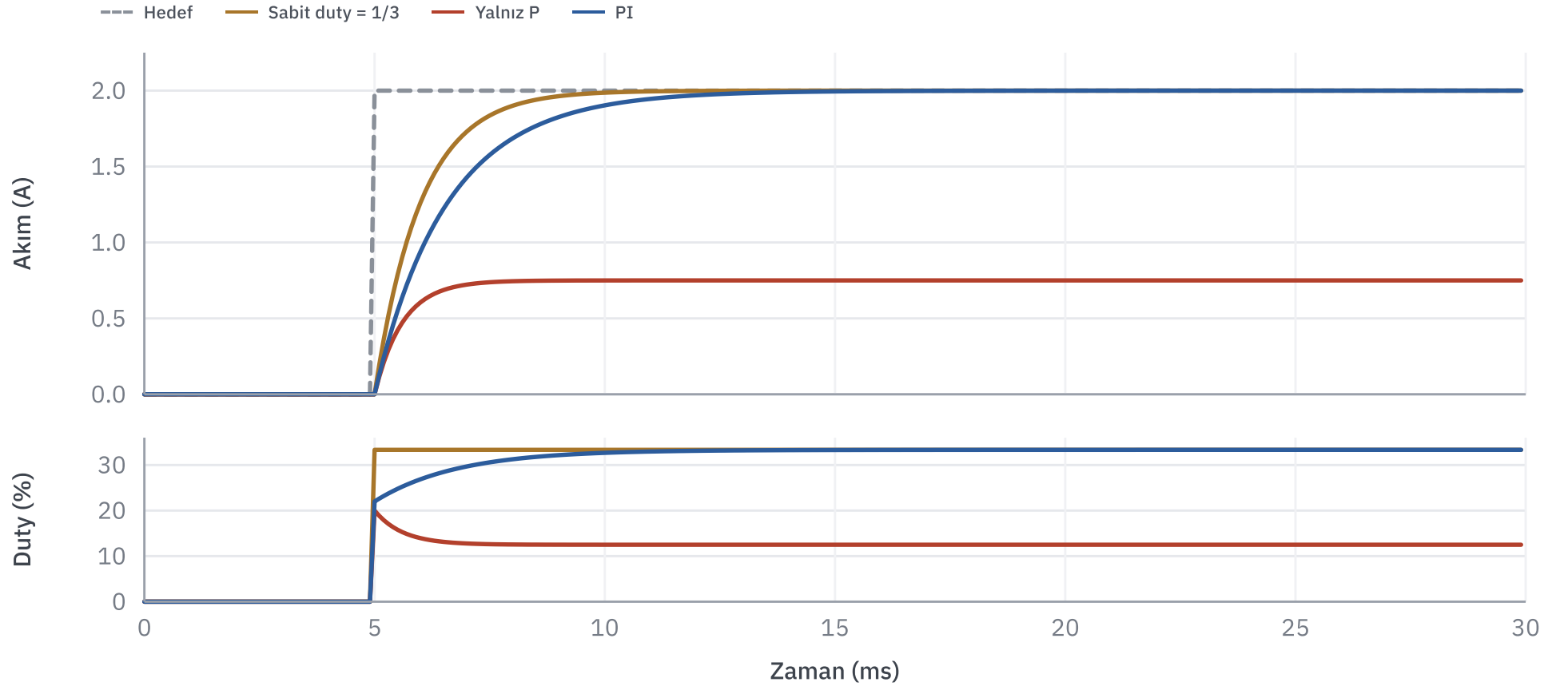
Duty dalgalanması: Gürültü deneyinde 80–160 ms aralığındaki duty'nin standart sapması; yüzde puan olarak raporlanır.

Değişiklik yaparken nereden başla?

PID sınıfında kazançlar; dosyanın başında devre parametreleri; `profile()` içinde hedef ve direnç değişimleri bulunur. Zaman sınırları adım indeksidir: $k=50$, 5 ms demektir. DT değişirse bu zamanlar da değişir.

Sabit duty, P ve PI karşılaştırması

5 ms anında hedef 0 A'dan 2 A'ya geçiyor. Direnç sabit 2Ω . Açık çevrim deneyinde duty doğrudan 1/3 yapılıyor; P ve PI deneyleri ölçülen akımla duty hesaplıyor.



YÖNTEM	SON AKIM	KALICI HATA	±%2 YERLEŞME
Sabit duty 1/3	2.000 A	Yaklaşık 0 A	4.0 ms
P (Kp = 0.10)	0.750 A	1.250 A	Yerleşmiyor
PI (0.10, 100)	2.000 A	Yaklaşık 0 A	6.6 ms

Süreler 5 ms'deki hedef değişiminden itibaren hesaplanmıştır. İki hedefe ulaşan eğride bu deneyde aşım %0'dır.

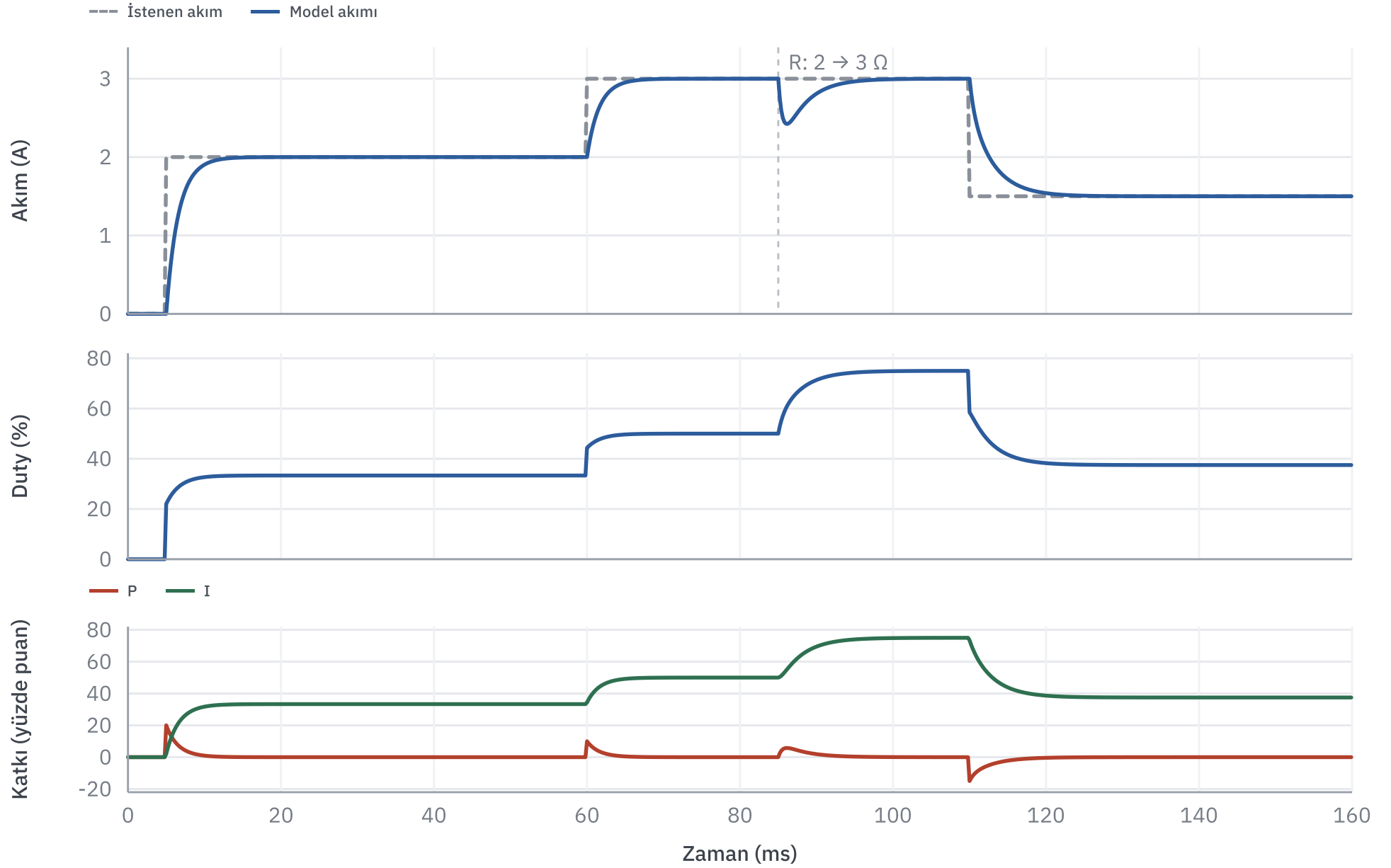
Grafikten çıkarılacak sonuç

Sabit duty bu ideal, parametreleri bilinen yükte hızlıdır. Bu sonuç kapalı çevrimin gereksiz olduğunu göstermez: sabit duty, değişen yükü ölçüp düzeltmez. PI'nin görevi yalnız hızlı yükselmek değil, ölçümle hedef arasındaki hatayı düzeltmektir.

Yalnız P eğrisi 0.75 A'da kalıyor; bölüm 06'daki cebirsel hesapla aynı sonuç. PI'de P katkısı azalırken integral katkısı gerekli sürekli duty'yi devralıyor.

Hedef ve direnç deęişince ne oluyor?

ZAMAN	DEęİŐİKLİK	BEKLENEN YENİ DENGE
5 ms	Hedef 2 A	$d = 2 \times 2/12 = 0.3333$
60 ms	Hedef 3 A	$d = 3 \times 2/12 = 0.50$
85 ms	Direnç $2 \rightarrow 3 \Omega$	3 A için $d = 0.75$
110 ms	Hedef 1.5 A	$d = 1.5 \times 3/12 = 0.375$

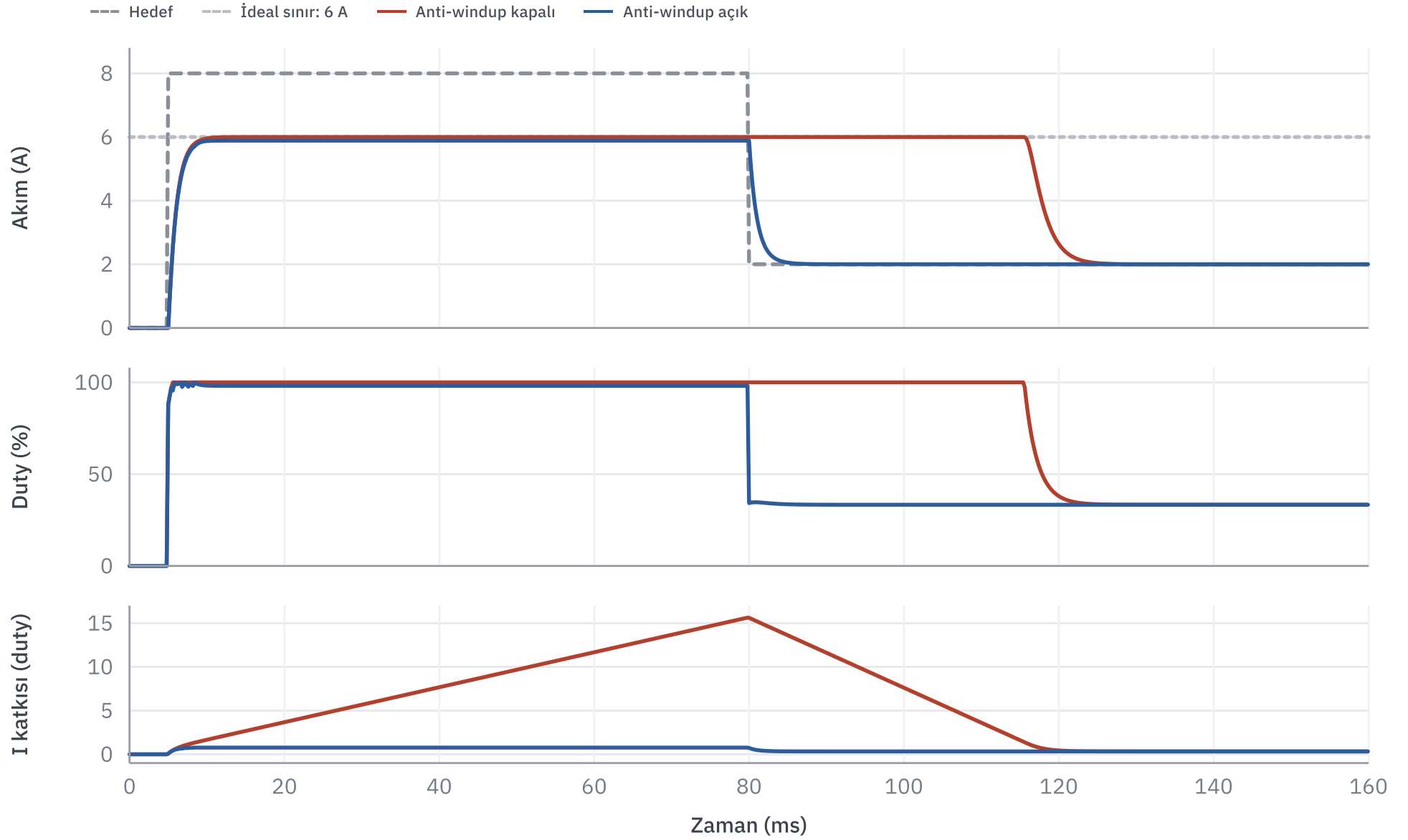


85 ms'yi oku: Direnç artınca mevcut duty yetersiz kalır ve akım düşer. Hata pozitif döner; P ve integral duty'yi yükseltir. Akım tekrar 3 A'ya yaklaşırken P sıfıra gider, integral yeni duty'yi taşır.

110 ms'yi oku: Hedef düşer. Hata negatif olur, duty azalır. Akım endüktans nedeniyle hemen sıfırlanmaz; yeni 1.5 A hedefine yaklaşır. Son akım yaklaşık 1.500 A'dır. D bu deney boyunca sıfırdır.

Hedef 8 A olursa: doyum ve integral

5-80 ms arasında 8 A istiyoruz. Devre en fazla 6 A'ya yaklaşabilir. 80 ms'de hedefi tekrar 2 A'ya indiriyoruz. İki kontrolcü arasındaki tek fark anti-windup seçeneğidir.



Kırmızı eğri: Akım 6 A civarındayken hata hâlâ 2 A. Her adımda integral yaklaşık $100 \times 2 \times 0.0001 = 0.02$ artıyor. Gerçek duty zaten %100; birikmenin devreye ek etkisi yok. Hedef düşünce büyük integralin geri azalması zaman alıyor.

Mavi eğri: Koşullu integral güncellemesi, çıkışı doyumun dışına iten birikmeyi engelliyor. Hedef 2 A'ya düşünce kontrol daha hızlı toparlıyor.^[2]

Sadece clip uygulamak aynı şey değil

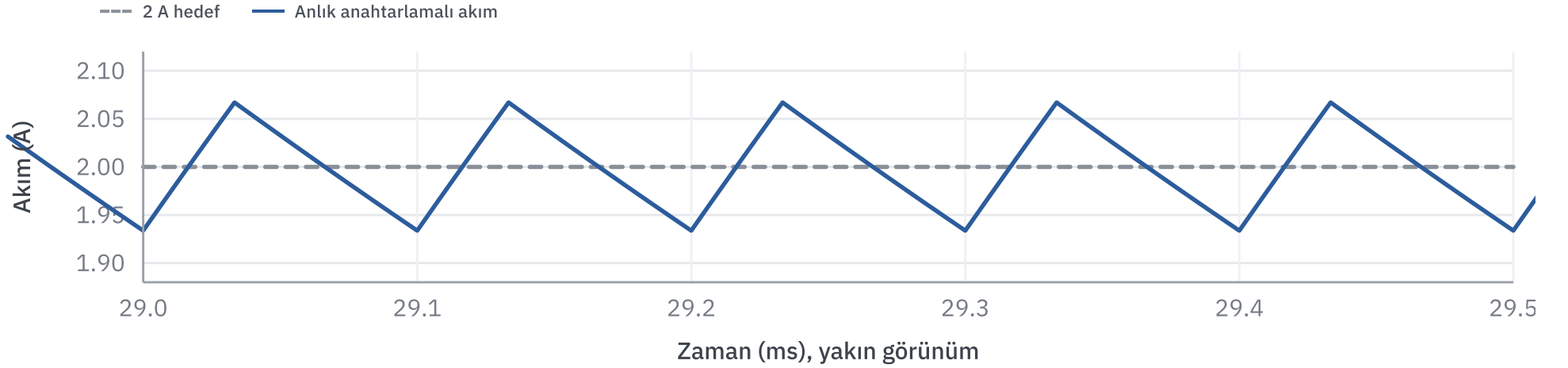
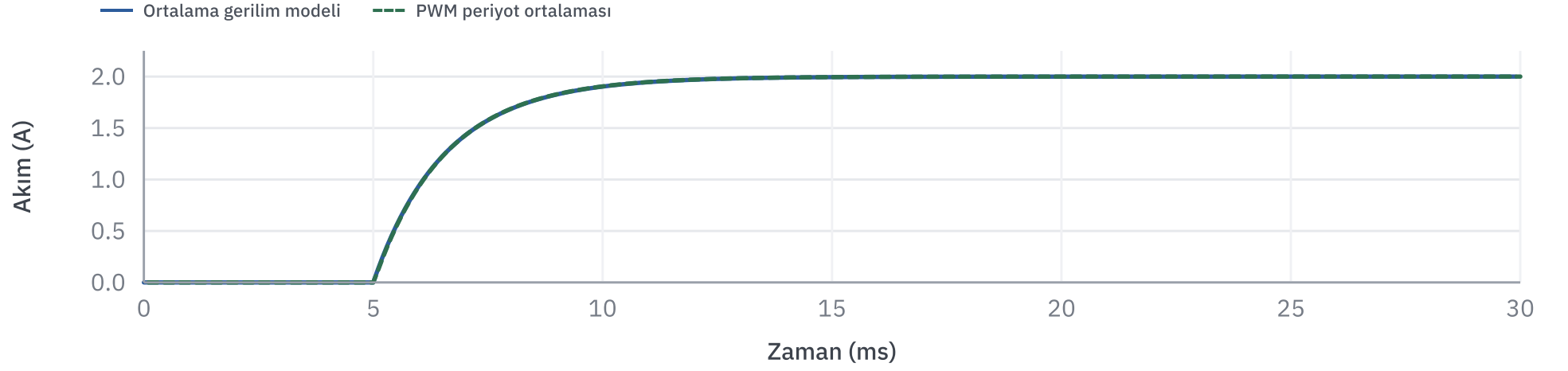
clip uygulanacak duty'yi sınırlar. Anti-windup ise içeride saklanan integral durumunu yönetir. Alttaki grafikte integral katkısının 1'i aşabilmesi, bunun uygulanmış duty olmadığını gösterir.

Ortalama modelden gerçek PWM darbelerine

Ortalama model tek adımda $v = d V_{bus}$ uygular. Anahtarlama model ise aynı periyodu iki parçaya ayırır:

$$i_{tepe} = F(i_k, V_{bus}, dT), \quad i_{k+1} = F(i_{tepe}, 0, (1-d)T).$$

Burada $F(i, v, t)$, bölüm 03'teki RL güncellemesidir. Kodda `segment()` ayrıca akımın zaman integralini hesaplar; ON ve OFF alanları toplamı T 'ye bölünerek periyot ortalaması bulunur.



Üst grafik: İki model benzer ortalama cevap verir. Anahtarlama deneyde kontrolcü, bir önceki tamamlanan PWM periyodunun akım ortalamasını kullanır. Bu yüzden ideal anlık geri beslemeye göre küçük bir ölçüm gecikmesi vardır.

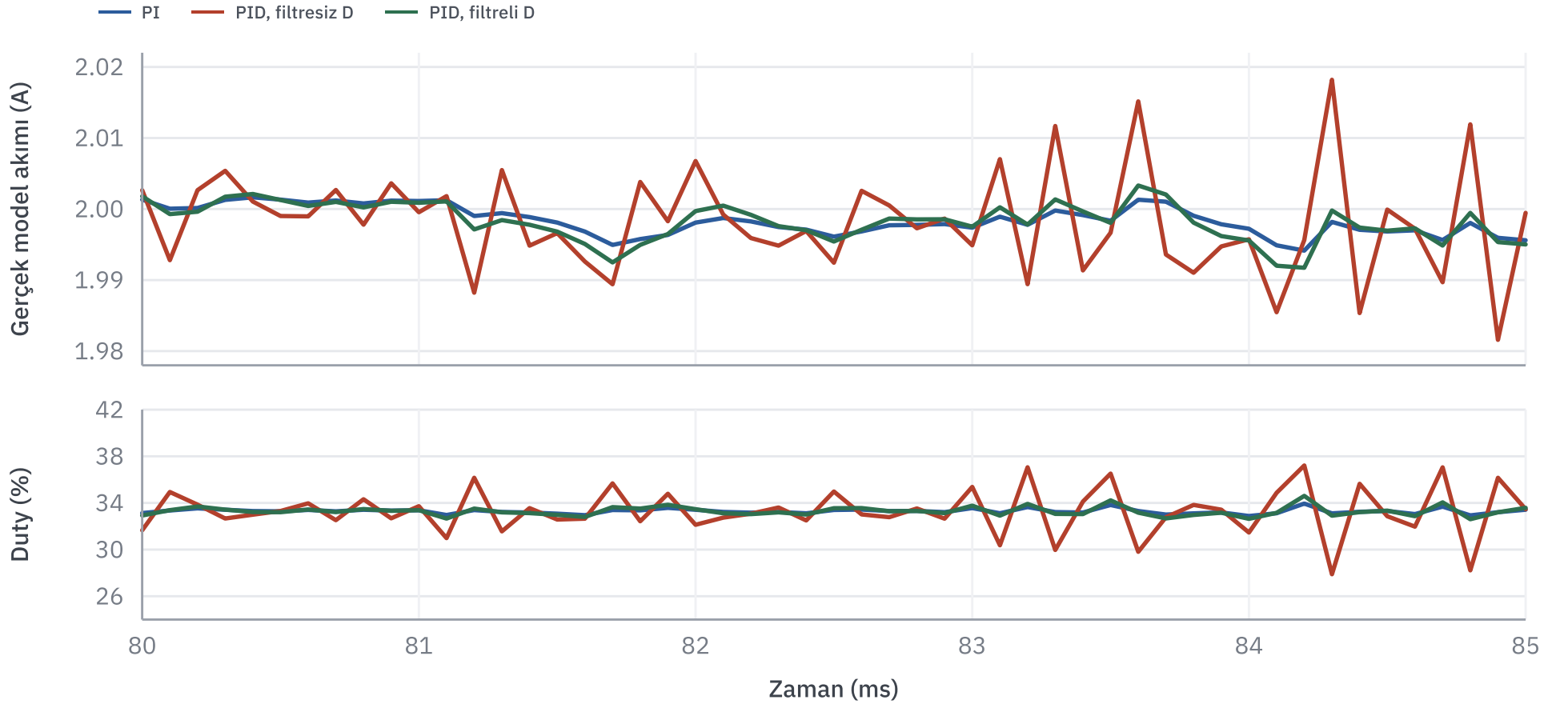
Alt grafik: Akım 2 A çevresinde dalgalanır. ON sırasında yükselir, OFF sırasında azalır. 2 A hedefini takip etmek, her mikrosaniyede tam 2 A olmak demek değildir.

Bu ölçüm ideal bir simülasyon tercihidir

Periyot ortalaması analitik olarak hesaplanır. Gerçek bir ADC'nin tek örneği otomatik olarak bu ortalamaaya eşit değildir. Anahtarlama anlarıyla uyumlu ölçüm ve sensör davranışı ayrıca tasarlanır. Bu yaklaşım tepe akım kontrollü bir PWM karşılaştırıcı devresi değildir.

D terimi akım ölçümünde ne yapar?

Ana deneyde PI yeterliydi. D'yi anlamak için ölçüme 0.02 A standart sapmalı, sıfır ortalamalı rastgele gürültü ekliyoruz. Üç koşulda aynı tohumla aynı gürültü dizisi kullanılıyor. Bu deney ortalama devre modeli üzerindedir.



KONTROLCÜ	D AYARI	DUTY STD. SAPMASI
PI	Kd = 0	0.226 yüzde puan
PID, filtresiz D	Kd = 0.00005, τD = 0	1.929 yüzde puan
PID, filtreli D	Kd = 0.00005, τD = 0.5 ms	0.400 yüzde puan

Ölçüm 1.99 A'dan 2.01 A'ya 100 µs'de çıkarsa filtresiz türev katkısı:

$$D = -0.00005 \frac{2.01 - 1.99}{0.0001} = -0.01.$$

Yani yalnız 0.02 A ölçüm farkı duty'yi 1 yüzde puan azaltabilir. Filtre bu hızlı değişimleri yumuşatır, fakat gecikme de ekler.^[3]

Bu deneyin sonucu: D eklemek burada daha sakın bir çıkış üretmedi. Bu, D'nin her sistemde gereksiz olduğu anlamına gelmez; bu yük ve örnek ayarlar için başlangıç seçimimiz PI'dir. D, hedefin değil ölçümün türevi olarak hesaplandığından hedef basamağından doğrudan türev darbesi oluşmaz.

Python'daki hangi satır değişecek?

Python şu anda hem kontrolcüyü hem de devreyi çalıştırıyor. Gerçek devrede RL güncellemesini işlemci hesaplamaz; akımı fiziksel devre değiştirir. Yazılım akımı sensörden okur ve PWM duty'yi günceller.

GÖREV	PYTHON	STM32 TARAFINDA
Hedef	<code>target = 2.0</code>	Amper cinsinden referans
Ölçüm	Model akımı	Akım sensörü + ADC
Kontrol	<code>pid.update(...)</code>	Aynı ayrık PI mantığı
Çıkış	0-1 duty	Timer PWM compare değeri
Devre cevabı	<code>averaged_step(...)</code>	Güç devresi ve yük
Zamanlama	Sayısal <code>for</code> döngüsü	Periyodik, PWM ile uyumlu tetikleme

ADC sayısını ampere çevirmek

İdeal N bit ADC için $V_{ADC} \approx ADC_{kod} V_{ref} / (2^N - 1)$. Shunt ve yükselteç kullanılan örnek bir ölçüm zincirinde:

$$i_{ölç} = \frac{V_{ADC} - V_{offset}}{G R_{shunt}}.$$

Örneğin $R_{shunt} = 0.05 \Omega$, $G = 20$, $V_{offset} = 0$ ise 2 A, ideal olarak 2 V ADC girişine karşılık gelir. Bunlar açıklama değerleridir; gerçek ölçümde sensör ofseti ve kazancı belirlenir. Kontrolcüye ham ADC sayısı yerine amper cinsinden ölçüm verilir.

Zamanlama ve PWM ölçeği

Bu rehberin adımı 100 μ s'dir, 100 ms değildir; aralarında 1000 kat fark var. Python'da bunu beklemeden simüle edebildik. Donanımda örnekleme ve hesap zamanı gerçekten bu takvime uymalıdır.

Timer sayım biçimine göre duty, periyodun sayaç adımı sayısı ile ölçeklenerek compare değerine çevrilir. Kesin formül PWM modu, polarite ve uç değeri davranışına bağlıdır; burada belirli timer veya pin varsaymıyoruz.

Mevcut HALL_RAW yapısının yeri

Önceki hall_a/b/c bitleri rotor konum/komütasyon durumuydu; akım ölçümü değildir. Bu saf RL deneyinde Hall dizisi gerekmez. Daha sonra motora geçersen komütasyon bilgisinin yanında ayrı bir akım ölçümü gerekir.

Alıştırmalar ve kontrol cevapları

1. Duty'den denge akımına

12 V ve 2 Ω için duty 0.25 olursa kaç amper beklenir?

Cevap: $i_{denge} = 0.25 \times 12/2 = 1.5$ A. Bu denge değeridir; mevcut akım aynı anda 1.5 A olmak zorunda değildir.

2. İlk PI adımını değiştir

`baslangic.py` içinde hedefi 1 A yap. İlk duty ve sonraki akımı elle bul.

Cevap: $P = 0.10$, $J = 0.01$, $d = 0.11$. $i_{denge} = 0.66$ A ve ilk yeni akım $0.095163 \times 0.66 \approx 0.062807$ A.

3. Hedefe ulaşıldığında integral ne olur?

2 A, $R = 2 \Omega$ ve sıfır hata durumunda `i_term` sıfırlanmalı mı?

Cevap: Hayır. Yaklaşık 0.333333 olarak kalır ve gerekli duty'yi sağlar. O adımda eklenen $K_i e \Delta t$ sıfırdır.

4. Direnci artır

2 A hedefi sabitken direnç 3 Ω olursa hangi duty gerekir? Hedef 5 A olursa ne olur?

Cevap: 2 A için duty 0.50. 5 A için 1.25 gerekir; mümkün değildir. Yeni ideal maksimum denge akımı $12/3 = 4$ A'dır.

5. Cevabı yavaşlat

$K_p = 0.05$, $K_i = 50$ yapıp yeniden çalıştır. Hata, duty ve yerleşme süresini karşılaştır. Sonra başlangıç ayarlarına dön.

Beklenti: Aynı ideal sürekli modelde tasarlanan bant genişliği yarıya iner, cevap yavaşlar. Yerleşme süresini tahmin etmek yerine CSV'den hesapla. Paket içindeki bazı doğrulamalar varsayılan parametrelere özeldir.

6. Sık görülen kod hatalarını yakala

HATA	NEDEN SORUN?
Her adımda <code>PID()</code> oluşturmak	Integral ve önceki ölçüm kaybolur.
<code>dt = 100</code> yazmak	100 μ s yerine 100 saniye kullanılır.
<code>duty = 50</code> yazmak	0-1 ölçeğinde bu %50 değildir.
<code>current = target</code> yazmak	Devre tepkisi ve kontrol problemi ortadan kalkar.
Sadece duty'yi kırpma	İntegral doyumda birikmeye devam edebilir.

Bu sonuçlar neyi doğruluyor?

Çalıştırılarak doğrulananlar

Python örnekleri yürütüldü. Ortalama modelin tek adımı analitik RL çözümüyle; yalnız P'nin son değeri 0.75 A hesabıyla karşılaştırıldı. PI'nin 2 A'ya ve yük değişimi deneyinin 1.5 A'ya yerleşmesi kontrol edildi. Duty sınırları, sıfır zaman adımının reddi ve doyum sonrası toparlanma da denetlendi.

Anahtarlama modelde %50 duty ile periyot ortalaması 3 A; tepeden tepeye dalgalanma yaklaşık 0.149969 A çıktı. Kapalı çevrim anahtarlama örneğinde de periyot ortalaması 2 A'ya yaklaştı. Grafikler ve tablolardaki sayılar paket içindeki koddan üretilmiştir.

Modelin sınırı

Bu, pozitif akımlı ideal seri RL yükü modelidir. ON durumunda 12 V, OFF durumunda 0 V ve ideal dolaşım yolu kabul edilir. Kaynak iç direnci, anahtar/diyot gerilim düşümü, endüktans doyumu, sensör bant genişliği, ADC/PWM nicemlemesi ve gerçek donanım gecikmeleri yoktur.

LED, batarya veya çıkış kapasitörlü dönüştürücü kullanırsan yük/devre denklemini yeniden kurmak gerekir. Dönen motora uygularsan geri EMK ve kullanılan komütasyon yapısı da devreye girer. Sadece değişken adını RPM'den akıma çevirmek fiziksel modeli doğru yapmaz.

İleri okuma: birincil kaynaklar

- [1] Texas Instruments / Dave Wilson. *Teaching Your PI Controller to Behave, Part II*. RL akım modeli ve PI ayarı. Rehberdeki duty ölçeği dönüşümü ve sayısal örnek ayrıca türetilmiştir.
<https://www.ti.com/lit/pdf/SSZTCG4>
- [2] MathWorks. *Anti-Windup Control Using PID Controller Block*. Doyum, integral birikmesi ve anti-windup yaklaşımı.
mathworks.com / Anti-Windup Control Using PID Controller Block
- [3] MathWorks. *PID Controller*. Ayırık zaman denetimi ve filtreli türev yapıları. Buradaki ölçüm üzerinden D ve koşullu integral kodu açıkça verilmiştir.
<https://www.mathworks.com/help/simulink/slref/pidcontroller.html>

[4] Analog Devices / Frederik Dostal. *Selecting the Right Inductor Current Ripple*. Anahtarlmalı dönüştürücülerde endüktans ve akım dalgalanması üzerine ek okuma; bu rehberin RL yüküyle aynı devre değildir.

[analog.com / Selecting the Right Inductor Current Ripple](https://analog.com/Selecting-the-Right-Inductor-Current-Ripple)

İdeal RL devresi / sayısal simülasyon. Kaynak erişimi: 8 Eylül 2026. Diyagramlar, kod ve deney grafikleri bu çalışma için hazırlanmıştır. Birincil kaynak görselleri kopyalanmamıştır.